



Tecnológico de Monterrey

Reflexión Evidencia 2. Resaltador de sintaxis paralelo

Santiago Aguilar Mello A01754105

Vladimir Pineda Reyes A01786772

Daniela Jannet Gil González A01752908

9 de Junio del 2026

Implementación de métodos computacionales (Gpo 602)

Profesor Víctor Manuel de la Cueva Hernández

Hemos hecho el análisis de los tiempos de corrida, y con ello, logramos notar la importancia del paralelismo al ejecutar varios archivos. Inicialmente, al tan solo ejecutar un archivo (el mismo que la entrega anterior), el programa requiere llamar al paralelizar, se requiere arrancar 16 procesos dado el número de CPUs lógicas que tiene la máquina de Santiago (que termina costando 20 ms), distribuir el trabajo y juntarlo todo. Esto hace que se corre 4.5 veces más lento. Sin embargo, se nota claramente que al meter más archivos y tener una mayor carga, el benchmark cambia totalmente como se ve en la siguiente tabla:

Archivos	Secuencial (s)	Paralela (s)	Speedup
1	0.0046	0.0206	0.222
5	0.0186	0.0167	1.114
10	0.0353	0.0181	1.946
25	0.0923	0.0252	3.666
50	0.201	0.0498	4.037
100	0.404	0.0811	4.981
150	0.6105	0.1096	5.568
200	0.7932	0.1585	5.003
250	1.0342	0.1731	5.976
300	1.2621	0.214	5.899

Aquí, se puede ver que el punto donde la corrida secuencial y paralela comienzan a balancearse es cuando llegan entre los 5 y 10 archivos porque el costo de comenzar a arrancar los procesos diferentes conviene al ya utilizar sus CPUs diferente y a partir de los 150 archivos, el speedup ya se estabiliza entre 5 y 6. Al pasar de 50 a 100 archivos el tiempo secuencial va de 0.201s a 0.404s, confirmando el crecimiento lineal

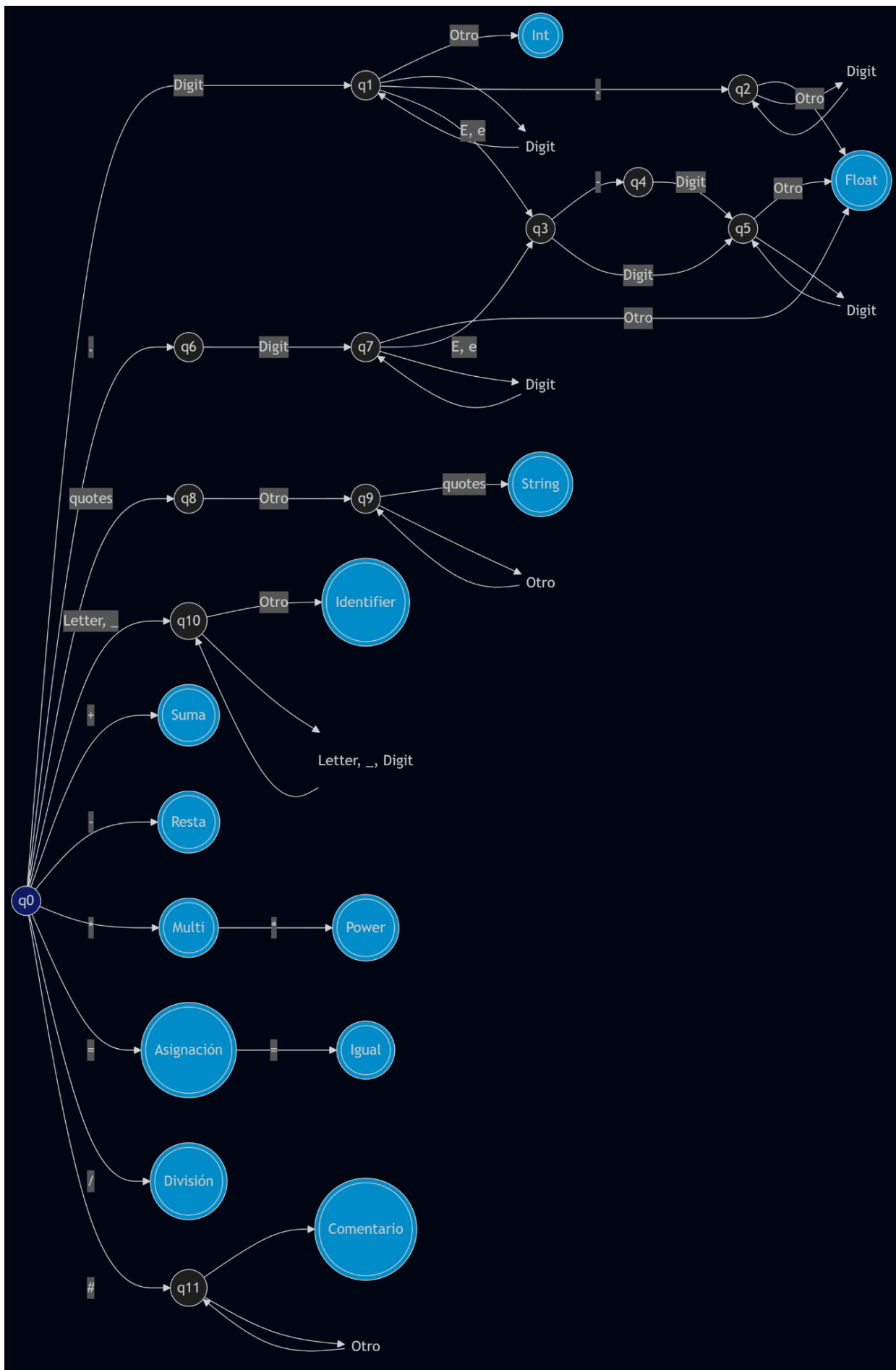
También se generó una segunda tabla, donde podemos ver las repeticiones con 300 archivos.

En esta tabla, los tiempos secuenciales llegan a variar más (1.35-1.60s) que los paralelos

(0.27 - 0.31s). El paralelo termina siendo más estable porque la carga se distribuye entre los núcleos de la CPU. Esto demuestra que nuestro programa da un speedup promedio de 5.171x usando 16 núcleos, donde la eficiencia real es del 32.3%

Corrida	Secuencial (s)	Paralela (s)	Speedup
1	1.3485	0.2881	4.68
2	1.5969	0.2723	5.865
3	1.4819	0.3096	4.787
4	1.5078	0.2909	5.183
5	1.521	0.2809	5.414
Promedio	1.4912	0.2884	5.171
Archivos	300		
Núcleos	16		

Dentro de la complejidad de nuestro algoritmo, el DFA que generamos tiene una complejidad de $O(n)$, ya que procesa la cadena de entrada realizando una iteración por carácter y clasifica el símbolo con la matriz de transiciones, haciendo que el tiempo de ejecución solo escale de forma lineal con el tamaño total del texto que tenemos que analizar.



Una implicación ética que el analizador lexer podría tener en la sociedad es que esta misma herramienta pueden usar los “hackers” para escanear contraseñas dentro de una base de datos. Inclusive, como se ha hecho varias veces en la vida real, se podría usar para escanear proyectos dentro de GitHub en busca de llaves API de empresas de IA para utilizar el token ajeno a su beneficio. Por otro lado, el paralelismo dentro del ámbito ético también tiene un impacto, ya que al optimizar nuestros programas con paralelismo y otras estrategias, podemos reducir la huella de carbono. El análisis de este reporte también demuestra que la optimización no es tan solo hacer paralelismo, sino también saber cuando. Si queremos reducir nuestra huella de carbono y requerimos tan solo leer un input pequeño, el paralelismo terminará haciendo que la computadora consume más recursos a comparación de tan solo hacerlo secuencialmente, pero el paralelismo también tiene su beneficio en el caso alterno donde hay un gran input.